

DSA Revision Notes

Topic 2 — Two Pointers & Sliding Window

Complete theory · All patterns · Complexity analysis
C++ code for every technique · Visual diagrams · LeetCode prep

10

Sections

20+

C++ Examples

15+

Patterns

8

Visuals

TOC Table of Contents**1. Introduction — What are Two Pointers & Sliding Window?**

→ Core idea, When to use, Problem signals

2. Two Pointers — Opposite Ends Pattern

→ Template, Sorted array pairs, Container With Most Water

→ Trapping Rain Water, 3Sum, 4Sum

3. Two Pointers — Same Direction (Fast/Slow)

→ Remove duplicates, Move zeroes, Partition

→ Is Subsequence, Merge sorted arrays

4. Two Pointers — Dutch National Flag

→ Sort Colors, 3-way partition template

5. Two Pointers — Palindrome Patterns

→ Valid Palindrome, Valid Palindrome II

6. Sliding Window — Fixed Size

→ Template, Max sum of size K, First negative in window

7. Sliding Window — Variable Size (Expand/Shrink)

→ Template, Longest substring without repeat

→ Min Window Substring, Permutation in String

8. Sliding Window — At-Most-K Trick

→ Exactly K = $\text{AtMost}(K) - \text{AtMost}(K-1)$

→ Subarrays with K different integers

9. Advanced Patterns

→ Sliding Window Maximum (Monotonic Deque)

→ Longest Repeating Character Replacement

10. Complexity & Pattern Cheat Sheet

1 Introduction

1.1 What are Two Pointers?

The **Two Pointer** technique uses two index variables that traverse a data structure — usually an array or string — simultaneously. Instead of nested loops ($O(n^2)$), each pointer moves at most n steps total, giving $O(n)$ time with $O(1)$ extra space. It is one of the most powerful tricks for reducing brute-force solutions.

■ Three Two-Pointer Variants

Opposite ends: $l=0$, $r=n-1$, converge inward. Use on SORTED arrays for pair/triplet problems.

Same direction: slow and fast both move left-to-right. Use for partitioning or subsequences.

Fast/Slow (Floyd): one moves 1 step, other 2 steps. Use for cycle detection in linked lists.

1.2 What is Sliding Window?

A **Sliding Window** is a contiguous subarray/substring that 'slides' across the data. Instead of recomputing the window from scratch each step ($O(n^2)$), we add the new right element and remove the old left element — $O(1)$ per slide. Two types exist:

- **Fixed window:** window size k is constant. Slide by one position each step.
- **Variable window:** expand right freely; shrink left when window becomes invalid.

1.3 When to Use — Problem Signals

Signal in Problem	Likely Technique
"Find pair that sums to target"	Two Pointers (opposite ends) on sorted array
"Longest subarray/substring with..."	Sliding Window (variable)
"Subarray of size k with max sum"	Sliding Window (fixed)
"Remove duplicates in-place"	Two Pointers (same direction)
"Is S a subsequence of T ?"	Two Pointers (same direction)
"Minimum window containing all chars"	Sliding Window + frequency map
"Exactly K distinct / equal K "	AtMost(K) - AtMost($K-1$) trick
"Sort array with 3 values"	Dutch National Flag (three pointers)
"Container / trap water"	Two Pointers (opposite ends)
"Cycle in linked list"	Fast/Slow pointers (Floyd)

Table 1: Problem signal $\rightarrow$ technique mapping

C++

```
1  #include <vector>
2  using namespace std;
3
4  vector<int> twoSum(vector<int>& numbers, int target) {
5      int l = 0, r = numbers.size() - 1;
6      while (l < r) {
7          int sum = numbers[l] + numbers[r];
8          if (sum == target) return {l + 1, r + 1}; // 1-indexed
9          else if (sum < target) l++; // need bigger sum
10         else r--; // need smaller sum
11     }
12     return {}; // guaranteed to find per problem
13 }
14 // Time: O(n) Space: O(1)
```

2.3 Container With Most Water (LC 11)

Given heights of walls, find two walls that trap the most water. Area = $\min(\text{height}[l], \text{height}[r]) \times (r - l)$. **Greedy insight:** always move the pointer with the *shorter* wall — keeping the taller wall gives a chance for a larger area with a closer wall.

C++

```
1  int maxArea(vector<int>& height) {
2      int l = 0, r = height.size() - 1;
3      int best = 0;
4      while (l < r) {
5          int area = min(height[l], height[r]) * (r - l);
6          best = max(best, area);
7          // Move the shorter wall inward
8          if (height[l] < height[r]) l++;
9          else r--;
10     }
11     return best;
12 }
13 // Time: O(n) Space: O(1)
```

■ Proof of Greedy Choice

Why does moving the shorter wall work?

Current area is limited by $\min(h[l], h[r])$. Moving the TALLER wall can only make it worse or equal (width shrinks, height capped by shorter wall). Moving SHORTER gives a chance.

2.4 Trapping Rain Water (LC 42)

Water at index $i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$. Brute force: $O(n^2)$. Prefix arrays: $O(n)$ time $O(n)$ space. **Two pointers: $O(n)$ time $O(1)$ space.** Key insight: if $\text{maxLeft} < \text{maxRight}$, the water at i is determined by maxLeft alone — process i and move it inward.

```
C++
1  int trap(vector<int>& height) {
2      int l = 0, r = height.size() - 1;
3      int maxL = 0, maxR = 0, water = 0;
4      while (l < r) {
5          if (height[l] <= height[r]) {
6              // Left side is the bottleneck
7              if (height[l] >= maxL) maxL = height[l];
8              else water += maxL - height[l];
9              l++;
10         } else {
11             // Right side is the bottleneck
12             if (height[r] >= maxR) maxR = height[r];
13             else water += maxR - height[r];
14             r--;
15         }
16     }
17     return water;
18 }
19 // Time: O(n)   Space: O(1) — optimal solution
```

2.5 3Sum (LC 15)

Find all unique triplets that sum to zero. Strategy: **sort** the array, then for each element i , run two pointers on the remaining subarray $[i+1, n-1]$. Skip duplicates explicitly to avoid repeated triplets.

C++

```
1  vector<vector<int>> threeSum(vector<int>& nums) {
2      sort(nums.begin(), nums.end());
3      vector<vector<int>> res;
4      int n = nums.size();
5      for (int i = 0; i < n - 2; i++) {
6          if (i > 0 && nums[i] == nums[i-1]) continue; // skip dup
7          int l = i + 1, r = n - 1;
8          while (l < r) {
9              int sum = nums[i] + nums[l] + nums[r];
10             if (sum == 0) {
11                 res.push_back({nums[i], nums[l], nums[r]});
12                 while (l < r && nums[l] == nums[l+1]) l++; // skip dup
13                 while (l < r && nums[r] == nums[r-1]) r--; // skip dup
14                 l++; r--;
15             } else if (sum < 0) l++;
16             else r--;
17         }
18     }
19     return res;
20 }
21 // Time: O(n^2)   Space: O(1) extra (output not counted)
```


3.3 Move Zeroes (LC 283)

Move all zeroes to the end while preserving relative order of non-zero elements. Slow pointer is the insertion position for non-zeroes.

C++

```
1  void moveZeroes(vector<int>& nums) {
2      int slow = 0;
3      // Phase 1: copy all non-zeroes to front
4      for (int fast = 0; fast < nums.size(); fast++) {
5          if (nums[fast] != 0) {
6              nums[slow++] = nums[fast];
7          }
8      }
9      // Phase 2: fill rest with 0
10     while (slow < nums.size()) nums[slow++] = 0;
11 }
12
13 // Alternative: swap version (preserves relative order of 0s too)
14 void moveZeroesSwap(vector<int>& nums) {
15     int slow = 0;
16     for (int fast = 0; fast < nums.size(); fast++) {
17         if (nums[fast] != 0)
18             swap(nums[slow++], nums[fast]);
19     }
20 }
21 // Time: O(n)   Space: O(1)
```

3.4 Is Subsequence (LC 392)

Check if string s is a subsequence of t. Use two pointers: i on s, j on t. Advance j always; advance i only when t[j] == s[i]. If i reaches end of s → true.

C++

```
1  bool isSubsequence(string s, string t) {
2      int i = 0, j = 0;
3      while (i < s.size() && j < t.size()) {
4          if (s[i] == t[j]) i++; // matched one char of s
5          j++; // always advance t
6      }
7      return i == s.size(); // matched all of s?
8  }
9  // Time: O(|s| + |t|)   Space: O(1)
```

3.5 Merge Sorted Array (LC 88)

Merge nums2 into nums1 in-place. **Key trick: start from the end** to avoid overwriting. Three pointers: p1 at end of nums1's data, p2 at end of nums2, p at end of merged.

C++

```
1 void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
2     int p1 = m - 1;           // pointer in nums1 data
3     int p2 = n - 1;           // pointer in nums2
4     int p = m + n - 1;        // write position (end)
5     while (p1 >= 0 && p2 >= 0) {
6         if (nums1[p1] > nums2[p2])
7             nums1[p--] = nums1[p1--];
8         else
9             nums1[p--] = nums2[p2--];
10    }
11    // Copy remaining nums2 (nums1 leftover already in place)
12    while (p2 >= 0)
13        nums1[p--] = nums2[p2--];
14 }
15 // Time: O(m+n)   Space: O(1)
```


C++

```
1 void sortColors(vector<int>& nums) {
2     int lo = 0, mid = 0, hi = nums.size() - 1;
3     while (mid <= hi) {
4         if (nums[mid] == 0) {
5             swap(nums[lo++], nums[mid++]);
6         } else if (nums[mid] == 1) {
7             mid++;
8         } else { // nums[mid] == 2
9             swap(nums[mid], nums[hi--]);
10            // DON'T mid++ - swapped element needs checking
11        }
12    }
13 }
14 // Time: O(n) Space: O(1) - single pass only
```

■ Critical Insight

Why NOT increment mid after swapping with hi?

When we swap `arr[mid]` with `arr[hi]`, the new `arr[mid]` came from `hi` — it has not been examined yet.

If we incremented mid, we would skip checking this newly placed element.

But when we swap with lo, `arr[lo]` must have been 1 (since `mid >= lo` always), so it is safe to advance both.

5 Two Pointers — Palindrome Patterns

5.1 Valid Palindrome (LC 125)

A phrase is a palindrome if, after converting to lowercase and removing non-alphanumeric characters, it reads the same forward and backward. Classic opposite-ends two-pointer.

C++

```
1  #include <cctype>
2
3  bool isPalindrome(string s) {
4      int l = 0, r = s.size() - 1;
5      while (l < r) {
6          // Skip non-alphanumeric from both ends
7          while (l < r && !isalnum(s[l])) l++;
8          while (l < r && !isalnum(s[r])) r--;
9          if (tolower(s[l]) != tolower(s[r])) return false;
10         l++; r--;
11     }
12     return true;
13 }
14 // Time: O(n)   Space: O(1)
```

5.2 Valid Palindrome II (LC 680)

Can you make a string palindrome by removing **at most one** character? When pointers mismatch, try skipping either $s[l]$ or $s[r]$ and check if the rest is a palindrome.

C++

```
1  bool checkPalin(string& s, int l, int r) {
2      while (l < r) {
3          if (s[l] != s[r]) return false;
4          l++; r--;
5      }
6      return true;
7  }
8
9  bool validPalindrome(string s) {
10     int l = 0, r = s.size() - 1;
11     while (l < r) {
12         if (s[l] != s[r])
13             // Try deleting either character
14             return checkPalin(s, l+1, r) || checkPalin(s, l, r-1);
15         l++; r--;
16     }
17     return true; // already palindrome
18 }
19 // Time: O(n)   Space: O(1)
```


C++

```
1  vector<int> firstNegative(vector<int>& arr, int k) {
2      deque<int> dq;    // stores indices of negative numbers
3      vector<int> result;
4      int n = arr.size();
5      for (int r = 0; r < n; r++) {
6          // Add new negative to deque
7          if (arr[r] < 0) dq.push_back(r);
8          // Remove elements outside window
9          if (!dq.empty() && dq.front() <= r - k) dq.pop_front();
10         // Record answer once window is full
11         if (r >= k - 1) {
12             result.push_back(dq.empty() ? 0 : arr[dq.front()]);
13         }
14     }
15     return result;
16 }
17 // Time: O(n)   Space: O(k)
```

6.4 Count Occurrences of Anagrams (LC 438)

Fixed window of size `p.length()`. Use a frequency map. When window's freq map equals `p`'s freq map → anagram found.

C++

```
1  vector<int> findAnagrams(string s, string p) {
2      if (s.size() < p.size()) return {};
3      vector<int> pCount(26, 0), wCount(26, 0), result;
4      int k = p.size();
5      // Build freq maps for p and first window
6      for (int i = 0; i < k; i++) {
7          pCount[p[i] - 'a']++;
8          wCount[s[i] - 'a']++;
9      }
10     if (pCount == wCount) result.push_back(0);
11     // Slide window
12     for (int r = k; r < s.size(); r++) {
13         wCount[s[r] - 'a']++;    // add right
14         wCount[s[r - k] - 'a']--; // remove left
15         if (pCount == wCount) result.push_back(r - k + 1);
16     }
17     return result;
18 }
19 // Time: O(n)   Space: O(1) – fixed 26-char arrays
```


C++

```
1  int lengthOfLongestSubstring(string s) {
2      unordered_map<char, int> lastSeen; // char -> last seen index
3      int l = 0, best = 0;
4      for (int r = 0; r < s.size(); r++) {
5          // If char was seen and is inside current window
6          if (lastSeen.count(s[r]) && lastSeen[s[r]] >= l) {
7              l = lastSeen[s[r]] + 1; // jump l past the duplicate
8          }
9          lastSeen[s[r]] = r;
10         best = max(best, r - l + 1);
11     }
12     return best;
13 }
14 // Time: O(n)   Space: O(min(n, charset_size))
```

■ Two Implementations

Alternative using set: add s[r], while s[r] in set -> remove s[l], l++

The map version is faster: jump l directly to lastSeen[s[r]]+1 without one-by-one shrinking

Edge case: only jump l if lastSeen[s[r]] >= l (char was seen but before current window is OK)

7.3 Minimum Window Substring (LC 76) — Hard

Find the smallest substring of s that contains all characters of t (with frequency). **Strategy:** use two frequency maps and a counter 'have' vs 'need'. Expand right until window is valid (have == need), then shrink left to minimise.

C++

```
1  string minWindow(string s, string t) {
2      if (t.empty()) return "";
3      unordered_map<char, int> need, have_map;
4      for (char c : t) need[c]++;
5      int have = 0, required = need.size(); // distinct chars needed
6      int l = 0, minLen = INT_MAX, minL = 0;
7      for (int r = 0; r < s.size(); r++) {
8          char c = s[r];
9          have_map[c]++;
10         // Check if this char's freq now satisfies 'need'
11         if (need.count(c) && have_map[c] == need[c]) have++;
12         // Shrink from left while window is valid
13         while (have == required) {
14             if (r - l + 1 < minLen) {
15                 minLen = r - l + 1;
16                 minL = l;
17             }
18             char lc = s[l];
19             have_map[lc]--;
20             if (need.count(lc) && have_map[lc] < need[lc]) have--;
21             l++;
22         }
23     }
24     return minLen == INT_MAX ? "" : s.substr(minL, minLen);
25 }
26 // Time: O(|s| + |t|)   Space: O(|t|)
```

7.4 Permutation in String (LC 567)

Check if any permutation of s1 exists as a substring of s2. Equivalent to: does any fixed window of size |s1| in s2 have the same freq map as s1?

C++

```
1  bool checkInclusion(string s1, string s2) {
2      if (s1.size() > s2.size()) return false;
3      vector<int> need(26, 0), window(26, 0);
4      int k = s1.size();
5      for (char c : s1) need[c - 'a']++;
6      // Build first window
7      for (int i = 0; i < k; i++) window[s2[i] - 'a']++;
8      if (need == window) return true;
9      // Slide
10     for (int r = k; r < s2.size(); r++) {
11         window[s2[r] - 'a']++;
12         window[s2[r - k] - 'a']--;
13         if (need == window) return true;
14     }
15     return false;
16 }
17 // Time: O(|s1| + |s2|)   Space: O(1) — fixed 26-element arrays
```

7.5 Longest Subarray of 1s After Deleting One Element (LC 1493)

At most one zero allowed in the window (the 'deleted' element). Window invalid when it contains more than one zero — shrink left.

C++

```
1  int longestSubarray(vector<int>& nums) {
2      int l = 0, zeros = 0, best = 0;
3      for (int r = 0; r < nums.size(); r++) {
4          if (nums[r] == 0) zeros++;
5          while (zeros > 1) {           // window invalid
6              if (nums[l] == 0) zeros--;
7              l++;
8          }
9          // -1 because one element must be deleted
10         best = max(best, r - l);    // window size - 1
11     }
12     return best;
13 }
14 // Time: O(n)   Space: O(1)
```

8

Sliding Window — At-Most-K Trick

8.1 The Core Identity

Many problems ask for subarrays with **exactly** K of something. Direct sliding window doesn't work for 'exactly' because we can't grow and shrink symmetrically. **The trick:**

■ **ExactlyK = AtMost(K) - AtMost(K-1)**

$\text{count}(\text{exactly } K) = \text{count}(\text{at most } K) - \text{count}(\text{at most } K-1)$

atMost(K) is easy: standard variable window. When distinct > K, shrink left.

Run atMost twice with K and K-1, subtract the results.

8.2 Subarrays with K Different Integers (LC 992)

C++

```
1  int atMost(vector<int>& nums, int k) {
2      unordered_map<int,int> freq;
3      int l = 0, count = 0;
4      for (int r = 0; r < nums.size(); r++) {
5          freq[nums[r]]++;
6          while (freq.size() > k) { // too many distinct
7              freq[nums[l]]--;
8              if (freq[nums[l]] == 0) freq.erase(nums[l]);
9              l++;
10         }
11         count += r - l + 1; // all subarrays ending at r with <=k distinct
12     }
13     return count;
14 }
15
16 int subarraysWithKDistinct(vector<int>& nums, int k) {
17     return atMost(nums, k) - atMost(nums, k - 1);
18 }
19 // Time: O(n)   Space: O(k)
```

8.3 Binary Subarrays With Sum (LC 930)

Exact sum S in binary array = atMost(S) - atMost(S-1).

C++

```
1  int atMostSum(vector<int>& nums, int goal) {
2      if (goal < 0) return 0;
3      int l = 0, windowSum = 0, count = 0;
4      for (int r = 0; r < nums.size(); r++) {
5          windowSum += nums[r];
6          while (windowSum > goal) windowSum -= nums[l++];
7          count += r - l + 1;
8      }
9      return count;
10 }
11
12 int numSubarraysWithSum(vector<int>& nums, int goal) {
13     return atMostSum(nums, goal) - atMostSum(nums, goal - 1);
14 }
15 // Time: O(n)   Space: O(1)
```

9 Advanced Patterns

9.1 Sliding Window Maximum (LC 239) — Monotonic Deque

Find the maximum of every window of size k . Brute force $O(nk)$. **Monotonic deque** approach: maintain a deque of indices in **decreasing** order of values. Front is always the current window maximum. $O(n)$ time, $O(k)$ space.

```
C++
1  #include <deque>
2
3  vector<int> maxSlidingWindow(vector<int>& nums, int k) {
4      deque<int> dq;    // stores INDICES, values are decreasing
5      vector<int> result;
6      int n = nums.size();
7      for (int r = 0; r < n; r++) {
8          // Remove elements outside window from front
9          while (!dq.empty() && dq.front() <= r - k)
10             dq.pop_front();
11         // Maintain decreasing order: remove smaller elements from back
12         while (!dq.empty() && nums[dq.back()] <= nums[r])
13             dq.pop_back();
14         dq.push_back(r);
15         // Window is full — record max (front of deque)
16         if (r >= k - 1)
17             result.push_back(nums[dq.front()]);
18     }
19     return result;
20 }
21 // Time: O(n)   Space: O(k)
```

■ Monotonic Deque Invariant

Deque invariant: indices in deque are always in increasing order (left to right in window).

Values at those indices are in DECREASING order (front = max).

New element: pop all back elements with smaller value — they can NEVER be the max while r is in window.

Front expiry: if $\text{dq.front()} \leq r - k$, it has left the window — pop it.

9.2 Longest Repeating Character Replacement (LC 424)

You can replace at most k characters. Find the longest substring where you can make all chars the same. **Key insight:** window is valid when $(\text{window_size} - \text{max_frequency}) \leq k$. We only need to track the max frequency, and it only needs to increase (not decrease) for optimal answer.

C++

```
1  int characterReplacement(string s, int k) {
2      vector<int> freq(26, 0);
3      int l = 0, maxFreq = 0, best = 0;
4      for (int r = 0; r < s.size(); r++) {
5          freq[s[r] - 'A']++;
6          maxFreq = max(maxFreq, freq[s[r] - 'A']);
7          // If window invalid: (size - maxFreq) > k
8          // NOTE: we only shrink by 1, never fully recompute maxFreq
9          if ((r - l + 1) - maxFreq > k) {
10             freq[s[l] - 'A']--;
11             l++;
12         }
13         best = max(best, r - l + 1);
14     }
15     return best;
16 }
17 // Time: O(n)   Space: O(1) - 26-letter alphabet
```

■ Why maxFreq Only Increases

Why not recompute maxFreq when shrinking? The answer only improves when maxFreq increases.

Even if the actual max freq drops after shrinking, we do not need a smaller window than best.

This is a key non-obvious optimization — the window size never decreases, it only slides.

9.3 Max Consecutive Ones III (LC 1004)

Flip at most k zeros to ones. Find the longest subarray of 1s. Window invalid when zeros in window exceed k.

C++

```
1  int longestOnes(vector<int>& nums, int k) {
2      int l = 0, zeros = 0, best = 0;
3      for (int r = 0; r < nums.size(); r++) {
4          if (nums[r] == 0) zeros++;
5          if (zeros > k) { // window invalid
6              if (nums[l] == 0) zeros--;
7              l++;
8          }
9          best = max(best, r - l + 1);
10     }
11     return best;
12 }
13 // Time: O(n)   Space: O(1)
```

9.4 Minimum Size Subarray Sum (LC 209)

Find shortest subarray with sum $\geq$ target. Variable window: expand until sum $\geq$ target, then shrink left to find the minimum length.

C++

```
1  int minSubArrayLen(int target, vector<int>& nums) {
2      int l = 0, windowSum = 0, minLen = INT_MAX;
3      for (int r = 0; r < nums.size(); r++) {
4          windowSum += nums[r];
5          while (windowSum >= target) {           // window is valid
6              minLen = min(minLen, r - l + 1);
7              windowSum -= nums[l];
8              l++;
9          }
10     }
11     return minLen == INT_MAX ? 0 : minLen;
12 }
13 // Time: O(n)   Space: O(1)
14
15 // O(n log n) alternative: prefix sum + binary search
16 // For each r, binary search for smallest l where prefix[r]-prefix[l] >= target
```

10 Complexity & Pattern Cheat Sheet

10.1 Two Pointers — All Patterns

Pattern	Setup	Key Condition	Time	Space	Problems
Opposite Ends	$l=0, r=n-1$	Sorted; sum comparison	$O(n)$	$O(1)$	LC 1,11,15,42,167
Same Direction	$slow=0, fast=0$	fast qualifies $\rightarrow$ write to slow	$O(n)$	$O(1)$	LC 26,27,88,283,392
Dutch National Flag	$lo=0, mid=0, hi=n-1$	3 groups; invariant zone	$O(n)$	$O(1)$	LC 75
Palindrome Check	$l=0, r=n-1$	Chars must match inward	$O(n)$	$O(1)$	LC 125,680
Fast/Slow (Cycle)	$slow=head, fast=head$	fast moves 2x; meet = cycle	$O(n)$	$O(1)$	LC 141,142,287
3Sum / kSum	Fix one; two-ptr on rest	Skip duplicates after sort	$O(n^2)$	$O(1)$	LC 15,18,259

Table 2: Two Pointer Patterns

10.2 Sliding Window — All Patterns

Pattern	Window Control	Data Structure	Time	Problems
Fixed size	Add $arr[r]$, remove $arr[r-k]$	Sum / freq array	$O(n)$	LC 438,567,643
Variable (maximize)	Expand r ; shrink l while invalid	Map / set / count	$O(n)$	LC 3,76,424
Variable (minimize)	Expand r ; shrink l while valid	Map / count	$O(n)$	LC 76,209
Exactly K trick	$atMost(K) - atMost(K-1)$	Map	$O(n)$	LC 930,992
Monotonic Deque	Maintain decreasing deque of indices	<code>deque<int></code>	$O(n)$	LC 239,862
Two-pass	Left pass + right pass combine	Two arrays	$O(n)$	LC 42,135

Table 3: Sliding Window Patterns

10.3 Decision Tree — Which Pattern to Choose?

C++

```

1 /*
2 * Problem asks for...                               Use...
3 * ████████████████████████████████████████████████████████████████████████████
4 * Pair/triplet sum in SORTED array → Two Pointers (opposite ends)
5 * Pair sum in UNSORTED array      → HashMap (not two pointers)
6 * Remove/partition elements in-place → Two Pointers (same direction)
7 * 3 groups sort (0,1,2 or similar) → Dutch National Flag
8 * Palindrome check                → Two Pointers (opposite ends)
9 * Subsequence matching            → Two Pointers (same direction)
10 *
11 * Fixed window size k                      → Sliding Window (fixed)
12 * Longest subarray satisfying X     → Sliding Window (variable, max)
13 * Shortest subarray satisfying X    → Sliding Window (variable, min)
14 * Exactly K distinct/sum           → atMost(K) - atMost(K-1)
15 * Max in every window of size k          → Monotonic Deque
16 * Window of chars = permutation    → Fixed window + freq array
17 */

```

10.4 Complete LeetCode Problem Map

#	Problem	Pattern	Difficulty
3	Longest Substring Without Repeating Chars	Sliding Window Variable	Medium
11	Container With Most Water	Opposite Ends	Medium
15	3Sum	Sort + Two Pointers	Medium
18	4Sum	Sort + Two Pointers	Medium
26	Remove Duplicates from Sorted Array	Same Direction	Easy
27	Remove Element	Same Direction	Easy
42	Trapping Rain Water	Opposite Ends	Hard
75	Sort Colors	Dutch National Flag	Medium
76	Minimum Window Substring	Sliding Window + Map	Hard
88	Merge Sorted Array	Three Pointers	Easy
125	Valid Palindrome	Opposite Ends	Easy
167	Two Sum II	Opposite Ends	Medium
209	Minimum Size Subarray Sum	Sliding Window Min	Medium
239	Sliding Window Maximum	Monotonic Deque	Hard
283	Move Zeroes	Same Direction	Easy
392	Is Subsequence	Same Direction	Easy

424	Longest Repeating Char Replacement	Sliding Window Variable	Medium
438	Find All Anagrams in a String	Fixed Window + Freq	Medium
567	Permutation in String	Fixed Window + Freq	Medium
680	Valid Palindrome II	Opposite Ends	Easy
930	Binary Subarrays With Sum	AtMost trick	Medium
992	Subarrays with K Different Integers	AtMost trick	Hard
1004	Max Consecutive Ones III	Sliding Window Variable	Medium
1493	Longest Subarray of 1s After Deleting One	Sliding Window Variable	Medium

Table 4: Complete LeetCode problem map for Two Pointers & Sliding Window

■ Golden Rules — Two Pointers & Sliding Window

1. Two Pointers on SORTED array → think opposite ends. On unsorted → think same direction or hashmap.
2. Sliding window state = what makes it VALID. Define invalid clearly before coding.
3. For minimum subarray: shrink while VALID (greedy: take smallest valid window at each r).
4. For maximum subarray: shrink while INVALID (expand as much as possible before shrinking).
5. AtMost(K) trick: whenever 'exactly K' doesn't fit a direct window, subtract two atMost calls.
6. Monotonic deque: always ask 'can this old element ever be the answer for a future window?'
7. When window is fixed size k: sliding formula is add $\text{arr}[r]$, remove $\text{arr}[r-k]$ every step.
8. Two pointers is $O(n)$; nested loop is $O(n^2)$. Always ask: can I make one pointer never go back?